В данном руководстве приведено пошаговое описание процесса создание проекта приложения для работы с БД.

Примечание. Во всех работах изложение ведется применительно к IDE Eclipse, При использовании другой IDE (например, Intellij IDEA) работа над проектом может отличаться в некоторых деталях (не в программном коде). Все особенности работы в другой IDE оставляются на самостоятельное изучение.

1. Создание проекта приложения

Для создания программного приложения создадим новый проект Maven. Maven – это система сборки. Она позволяет настроить работу с зависимостями (сторонними библиотеками) и автоматизировать сборку проекта.

Для создания проекта необходимо выполнить команду меню:

`File – New – Maven Project`

В первом окне мастера создания проекта оставим выбор флага

`Use default Workspace location`

и перейдем к следующему шагу, нажав ***Next*** (рис.1).

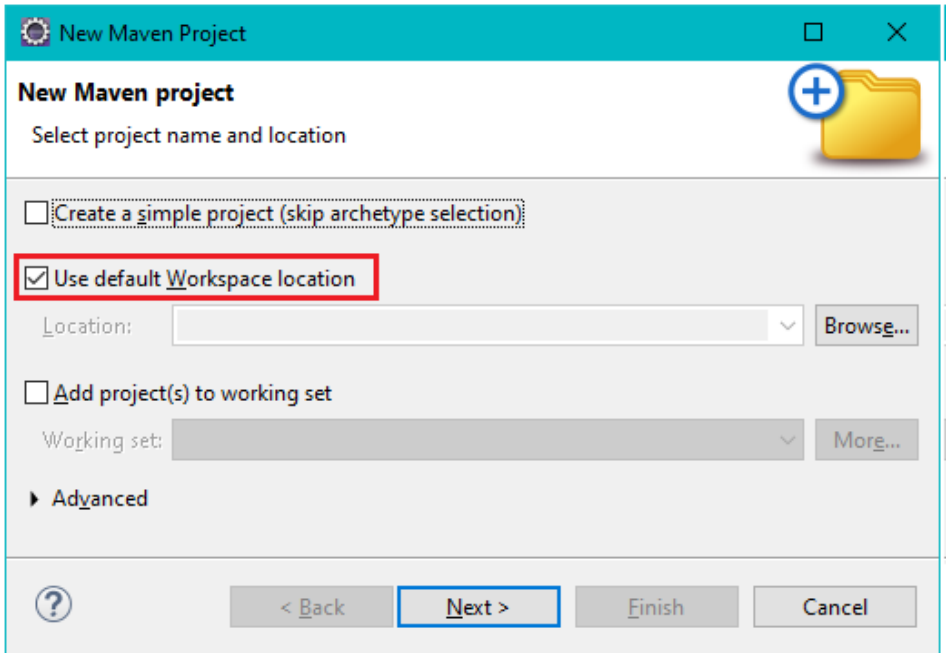


Рисунок 1. Выбор места размещения проекта по умолчанию

На следующем шаге выберем шаблон проекта (**`Archetype`**). Выберем опцию поиска во всех каталогах (**`All Catalog`**) и в строке **`Filter`** введем **`javafx`**. В списке шаблонов выберем **`javafx-archetype-fxml`** (из *Group Id* **`org.openjx`**) и нажмем кнопку **`Next`** (рис. 2).

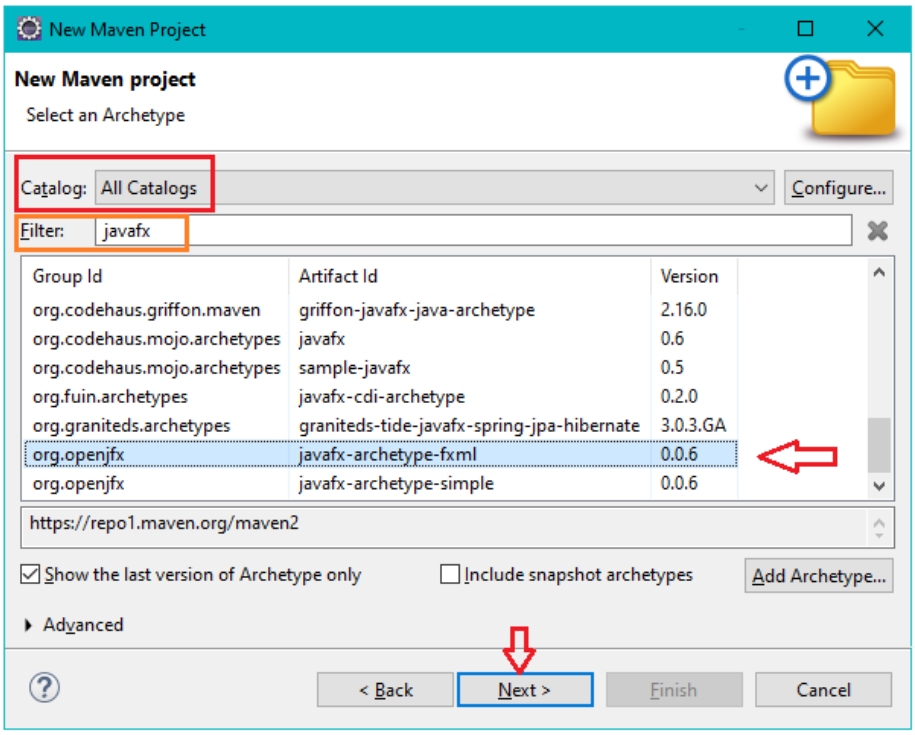


Рисунок 2. Выбор шаблона проекта

На завершающем шаге зададим имя проекта, имя корневого пакета (**`com`**), изменим версию JavaFX на 21 и запустим процесс создания проекта, нажав ***Finish*** (рис.3). Имена выбраны произвольно и могут быть изменены. Если установлена другая версия, укажите ее номер.

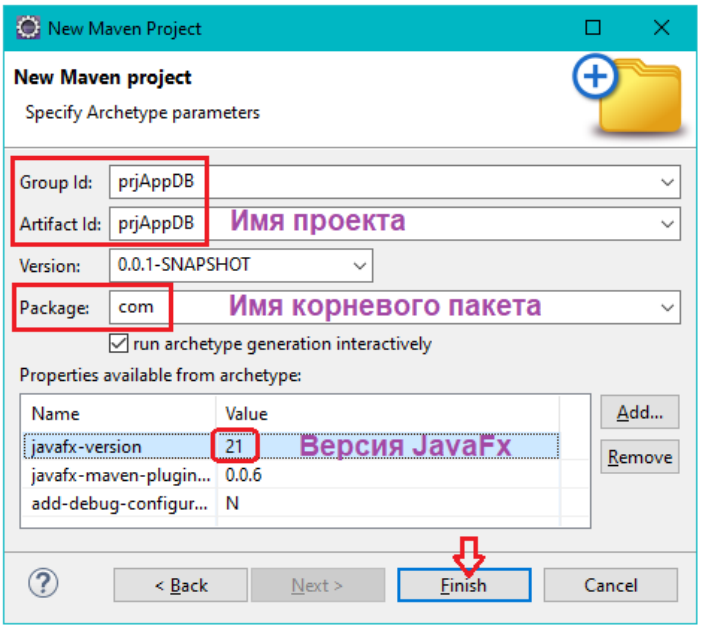


Рисунок 3. Задание параметров проекта

В консоли будет отображаться ход создания проекта и на завершающем шаге надо подтвердить создание, введя **`Y`** и нажав **`Enter`** (рис.4) .

```
Console ×   Progress
C:\Program Files\Java\jdk-21\bin\javaw.exe (16 нояб. 2024 г., 23:40:43) [pid: 13064]
[INFO] Using property: groupId = prjApp
[INFO] Using property: artifactId = prApp
[INFO] Using property: version = 0.0.1-SNAPSHOT
[INFO] Using property: package = com
Confirm properties configuration:
javafx-version: 21
javafx-maven-plugin-version: 0.0.6
add-debug-configuration: N
groupId: prjApp
artifactId: prApp
version: 0.0.1-SNAPSHOT
package: com
 Y: Y
```

Рисунок 4. Завершение процесса создания проекта

В дереве проектов будет добавлен новый проект (рис.5). В корневом пакете проекта com будет создан запускной класс **`App.java`**.

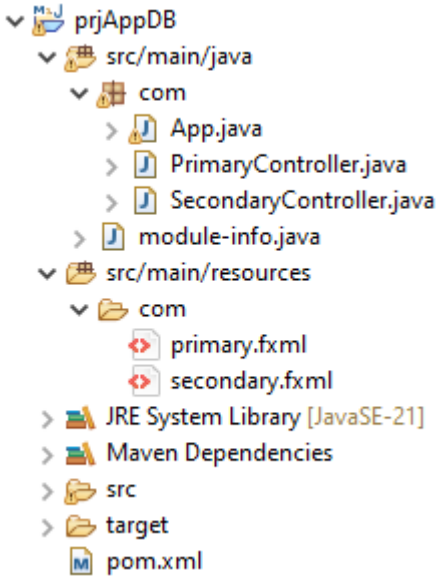


Рисунок 5. Структура нового проекта

Протестируем созданный проект. Для этого необходимо на запускном файле в контекстном меню выбрать пункт

`Run as – Java Application`

Будет выведено окно с кнопкой.

После тестирования переименуем класс **`App`** в **`MainApp`**. Также удалим в пакете **`com`** файлы классов контроллеров **`PrimaryController`** и **`SecondaryController`**, а в папке **`resources.com`** – файлы **`*.fxml`**.

Логику работы запускного файла `MainApp` изменим позже.

Обратите внимание, что в шаблоне проекта используется старая версия JRE 11. Откроем файл pom.xml и внесем в него изменения. Изменим версию Java на 21 в двух местах. Если установлена другая версия JDK, укажите ее номер.

```
<properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>21</maven.compiler.source>
```

```
    <maven.compiler.target>21</maven.compiler.target>
</properties>
```

и

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.8.0</version>
    <configuration>
          <release>21</release>
    </configuration>
</plugin>
```

Также добавим в конец блока `<dependencies>...</dependencies>` зависимость (описание внешней библиотеки) для драйвера PostgreSQL версии 42.7.5:

```
<dependencies>
    ...
  <dependency>
     <groupId>org.postgresql</groupId>
     <artifactId>postgresql</artifactId>
    <version>42.7.5</version>
   </dependency>
</dependencies>
```

Классы программного приложения целесообразно хранить в отдельных пакетах в соответствии с их назначением. Для этого необходимо в проекте создать новые папки и пакеты. Будем придерживаться следующей структуры проекта (рис. 6).

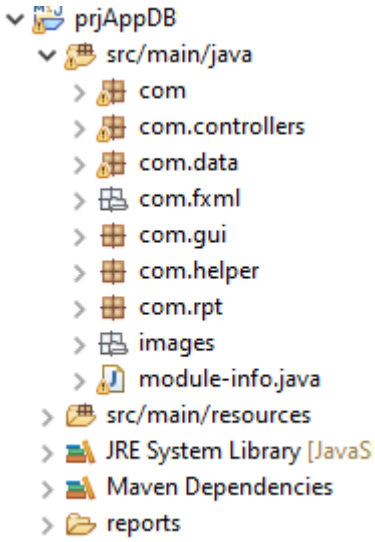


Рисунок 6. Пакеты и папки проекта

Отдельные пакеты (packages) и папки используются для хранения классов или файлов, сгруппированных по функциональному назначению.

В папке **`src/main/java`** содержатся:

- **основной пакет com**, в котором будем хранить основной класс приложения, класс взаимодействия с БД и файлы стилей CSS.
- папка **images**, в которой разместим файлы с изображениями.

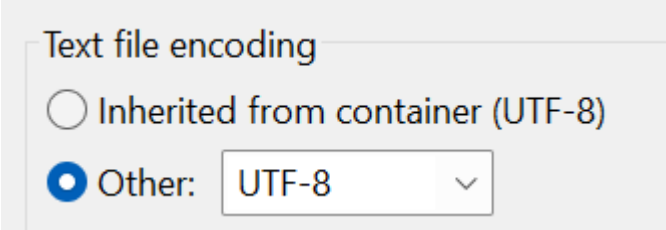
Пакет **com** включает папки и пакеты, которые содержат:

- **controllers** – классы контроллеров
- **data** – классы, описывающие данные объектов (они ассоциируются с таблицами БД);
- **fxml** – файлы описания пользовательского интерфейса (*.**fxml)**;
- **gui** – собственные классы пользовательского интерфейса, которые могут использоваться в основных классах (диалоги и т.д.);
- **helper** – вспомогательные классы, обеспечивающие конвертацию типов данных, обработку изображений и т.д.
- **rpt** – классы параметров и наборов данных для отчетов, обеспечивающие вывод отчетов, обработку изображений и т.д.

Добавим в проект в папке **src** все пакеты и папки проекта, начиная с пакета com, в котором будем создавать основные классы приложения. Дополнительно в проект добавим папку **reports**, которая будет содержать файлы шаблонов форм отчетов.

В качестве СУБД будем использовать PostgreSQL (целесообразно использовать версию не ниже 17).

Примечание. Кодировка кода проекта должна быть UTF8. Проверить это можно в свойствах проекта в разделе *Resource* в блоке *Text file encoding*



2. Программная логика основного класса приложения

Основной класс приложения (в руководстве он имеет имя **`MainApp`**) реализует логику входа в приложение. В классе инициируется вывод окна авторизации пользователя в БД. При успешной авторизации выводится основное окно приложения. Логика работы основного класса приведена на рисунке 7.

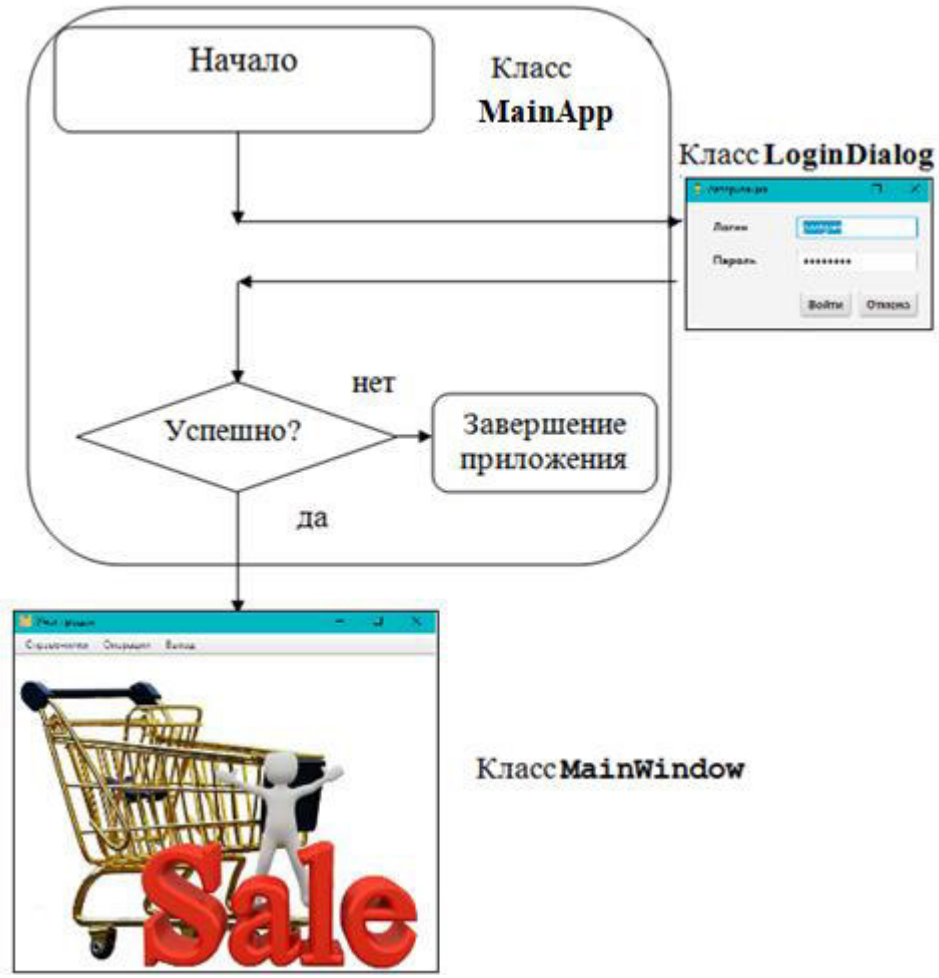


Рисунок 7. Логика работы основного класса MainApp

Рассмотрим основные этапы создания приложения:

Создание класса менеджера взаимодействия с БД DBManager
Создание файла properties (файлы свойств)
Создание класса с методами стандартных диалогов
Создание класса окна соединения с БД `LoginDialog`
Создание основного окна приложения `MainWindow`
Изменение основного класса `MainApp` и тестирование приложения
Создание исполняемого `jar`-файла и его тестирование

Выполнение этих этапов описано в п.3-9 данного руководства.

3. Создание класса менеджера взаимодействия с БД DBManager

Данный класс будет хранить поле, содержащее ссылку на установленное соединение, и содержать методы выполнения запросов к БД.

Поместим в класс метод для тестирования соединения. Метод выполняет запрос на получения версии БД.

```
package com

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
/*
 *  Менеджер взаимодействия с БД
*/
public class DBManager {
    private Connection con = null;
    public Connection getConnection(){
       return con;
    }
    public void setConnection(Connection con) {
       this.con=con;
    }
     // Метод тестирования соединения
     public String getVersion() {
          String ver=null;
          Statement stmt;
          ResultSet rset;
          try {
               stmt = con.createStatement();
               rset = stmt.executeQuery("SELECT VERSION()");
               while (rset.next())
               {
                    ver =rset.getString(1);
               }

          } catch (SQLException e) {
               e.printStackTrace();
          }
          return ver;
     }

}
```

В следующих работах в этот класс будем помещать методы для выполнения других запросов к БД.

В начало

4. Создание конфигурационного файла

Конфигурационный файл (файл свойств) предназначен для хранения параметров конфигурации приложения, которые желательно хранить вне программы. В нашем случае в файле будем хранить строку соединения с БД PostgreSQL.

Создадим файл **`conf.prop`** и положим его в корневую папку проекта. При создании исполняемого `jar`-файла файл свойств надо размещать рядом с исполняемым файлом. Это удобно для внесения правок в файл свойств для работающего приложения.

Выделим имя проекта и выполним пункт меню

`File – New – File`

и в открывшемся окне зададим имя файла **`conf.prop`** (рис.8).

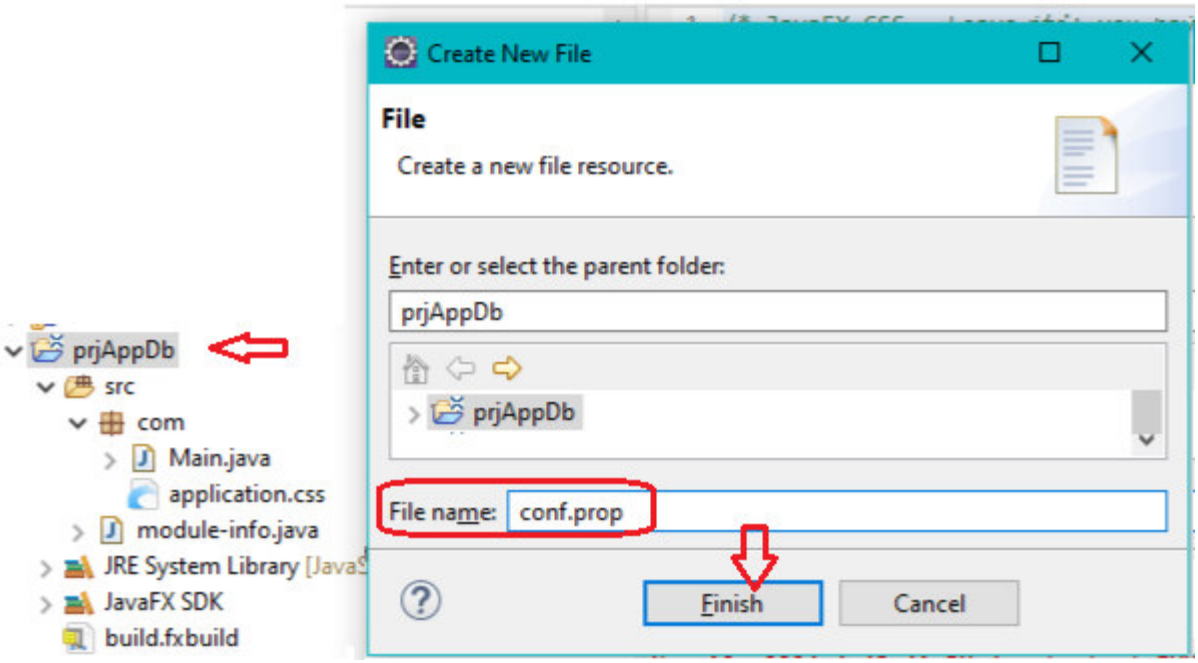


Рисунок 8. Создание файла свойств проекта

Содержимое файла представляет собой набор строк:

`ключ=значение`

Добавим в файл параметр для строки соединения с локальной БД (имя БД может быть другое):

`URL_DB=jdbc:postgresql://localhost:5432/test_db?currentSchema=prod`

где

`localhost` – сервер БД на локальном компьютере

5432 – порт, используемый сервером БД PostgreSQL (имя порта задается при установке PostgreSQL и его можно найти в PgAdmin в настройках сервера)

`test_db` – имя БД (надо указать имя БД, созданной в лабораторной работе 3).

`prod` – имя схемы.

Для чтение значения параметра из файла свойств необходимо вызвать метод `getProperty` класса `Properties`.

```
Объект_класса.getProperty("ключ")
```

Загрузку данных из файла свойств будем выполнять в методе инициализации класса контроллера `LoginDialogControl` (см п.6 данного документа).

На этапе разработки приложения, можно в файле свойств хранить логии и пароль пользователя, загружать эти свойства и устанавливать в поля ввода логина и пароля.

Например, чтение параметра «имя пользователя» (User) и запись его в поле ввода логина `txtLogin`:

```
txtLogin.setText(property.getProperty("User"));
```

Примечание. Имена свойств (параметров) являются регистрозависимыми. Кодировка файла должна быть UTF8. Если файл редактируется внутренним редактором, то это правило соблюдается.

В начало

5. Создание класса с методами стандартных диалогов

Создадим в пакете **com.gui** класс **Dialogs**, который содержит статические методы вывода диалоговых окон с информационными сообщениями и диалогами для подтверждением некоторого действия.

Метод вывода информационного диалога

```
public static void showDialog(String title, String message,
                                AlertType type) {
      Alert alert = new Alert(type);
      alert.setTitle(title);
      alert.setHeaderText(null);
      alert.setContentText(message);
      alert.showAndWait();
}
```

Метод вывода информационного диалога на базе некоторого окна

```
public static void showDialog(String title, String message,
                      AlertType type, Stage stage) {
      Alert alert = new Alert(type);
      alert.setTitle(title);
      if (stage!=null)
         alert.initOwner(stage);
      alert.setHeaderText(null);
      alert.setContentText(message);
      alert.showAndWait();
}
```

Метод вывода диалога подтверждения

```
public static boolean showConfirmDialog(String question, Stage stage){
      Alert alert = new Alert(AlertType.CONFIRMATION);
      alert.setTitle("Подтверждение");
      if (stage!=null)
         alert.initOwner(stage);
      alert.setHeaderText(null);
      alert.setContentText(question);
      Optional<ButtonType> result = alert.showAndWait();
      if (result.get() == ButtonType.OK) {
            return true;
      } else {
            return false;
      }
}}
```

При добавлении методов в класс компилятор подчеркнет красной волнистой чертой неразрешенные ссылки на используемые классы. Эти классы необходимо подключить директивой **import**. Импортировать классы можно автоматически. Для этого надо навести курсор мыши на подчеркнутое имя класса и в выпадающем меню выбрать строку **import ...** (рис. 9). В списке надо выбирать классы из пакетов **javafx.*.**

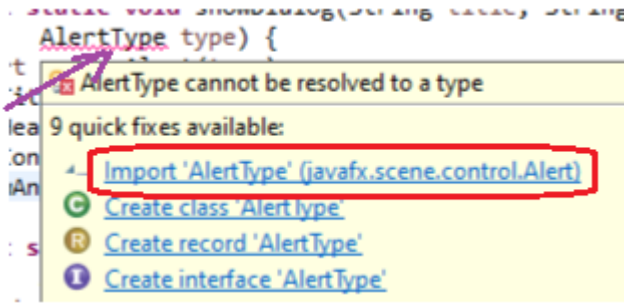


Рисунок 9. Импортирование используемых классов

6. Создание класса окна авторизации пользователя с БД

Все окна программного приложения представляются двумя файлами:

- файлом описания пользовательского интерфейса (GUI) на языке FXML;
- классом контроллера для взаимодействия пользователя с интерфейсом.

Для окна авторизации (рис.10) создадим 2 файла:

- файл описания GUI **`LoginDialog.fxml`** в пакете **`com.fxml`**

- класс контроллера **`LoginDialogController.java`** в пакете **`com.controllers`**.

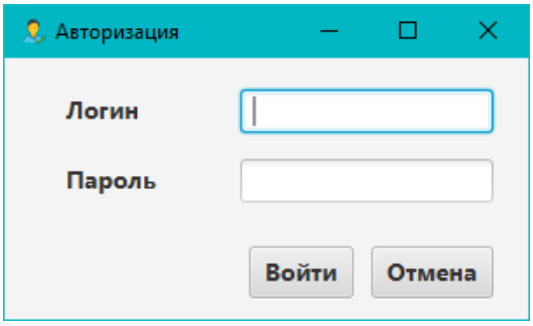


Рисунок 10. Окно авторизации

Создадим в пакете **`com.fxml`** файл FXML для описания пользовательского интерфейса, выполнив команду меню:

`File – New – Other`

и в группе **`JavaFX`** выберем **`New FXML Document`** (рис.11). Зададим имя файла **`LoginDialog`**. Корневую панель для размещения компонентов (`Root Element:` **`AnchorPane`**) оставим без изменения.

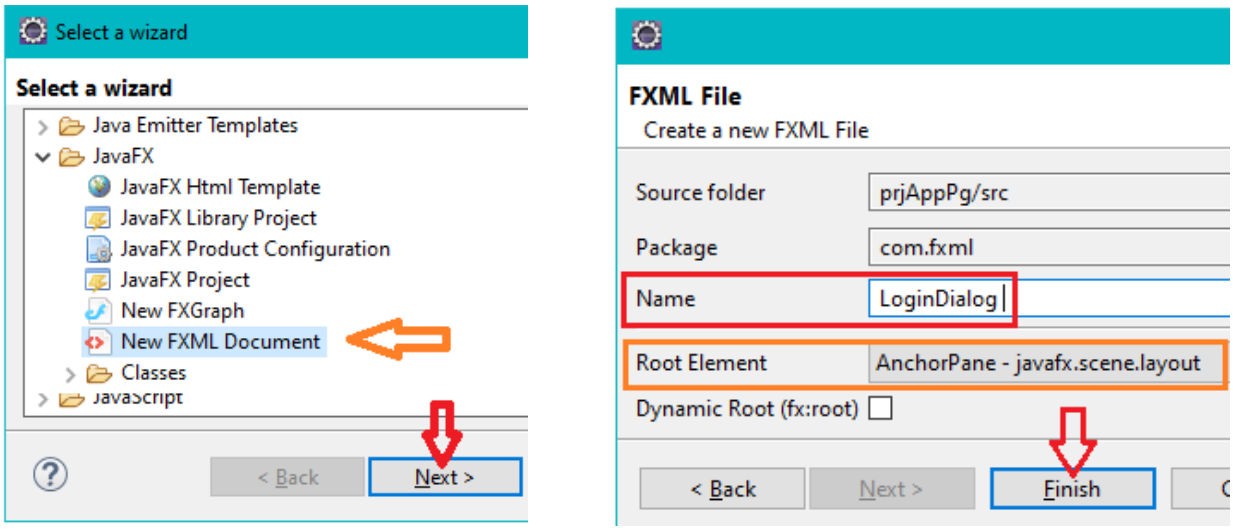


Рисунок 11. Создание файла FXML

Для редактирования файла откроем его в приложении Scene Builder (рис. 12).

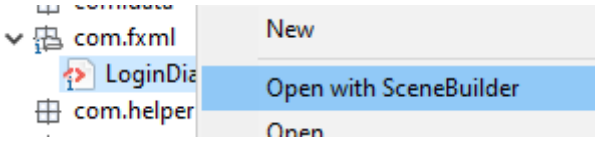


Рисунок 12. Открытие файла fxml в приложении Scene Builder

Установим фиксированный размер окна. Для этого в левой панели на вкладке **`Hierarchy`** выделим компонент **`AnchorPane`**, а затем на правой панели на вкладке **`Layout`** зададим параметры (рис.13):

`Pref Width=300` **`Pref Height=150`**

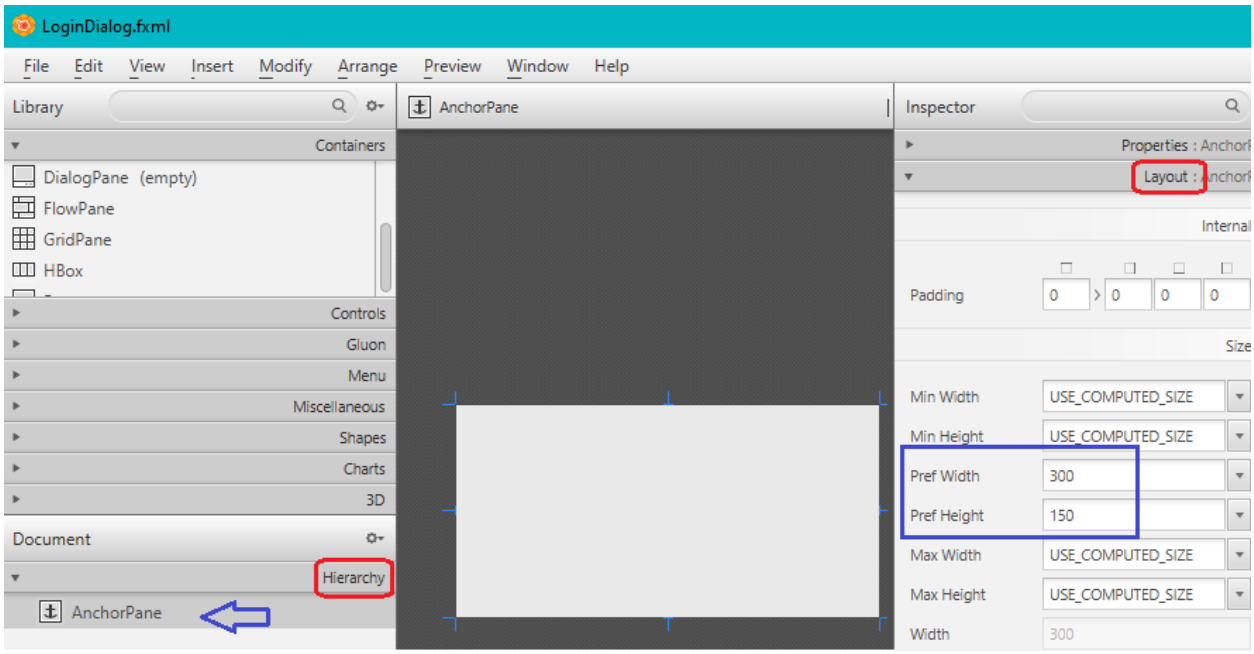


Рисунок 13. Задание размеров окна

Для размещения компонентов окна будем использовать контейнер-«сетку» **GridPane**. Для этого в левой панели с вкладки **Containers** перетащим компонент **GridPane** на вкладку **Hierarchy** на компонент **AnchorPane** (рис. 14).

Выделим и удалим третью строку контейнера-сетки (рис. 15). На правой панели на вкладке **Layout** зададим отступы контейнера-сетки от границ родительского контейнера **AnchorPane** в свойстве **Anchor Pane Constraints**: левый 25, правый 15, верхний 20 пикселей, а также расстояния между строками **Vgap=10** (рис. 16).

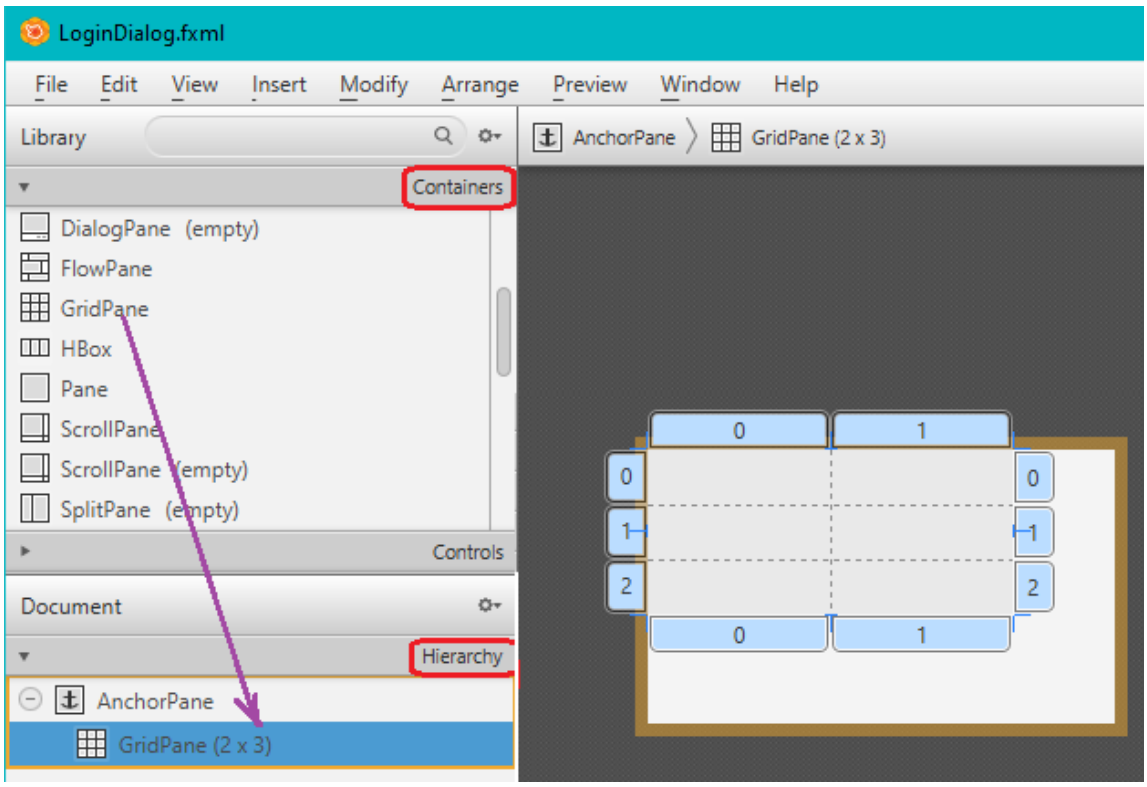


Рисунок 14. Размещение контейнера-сетки

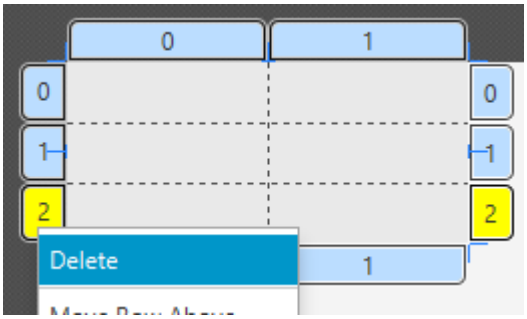


Рисунок 15. Удаление строки контейнера-сетки

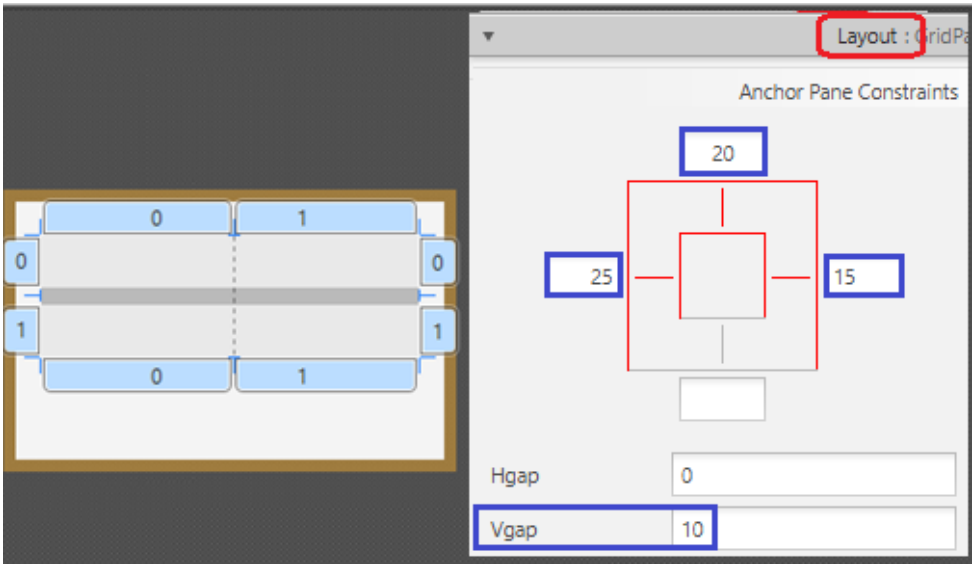


Рисунок 16. Установка отступов

Разместим компоненты **`Label`** перетащив их с вкладки **`Controls`** на левой панели в ячейки первой колонки контейнера-сетки (рис. 17).

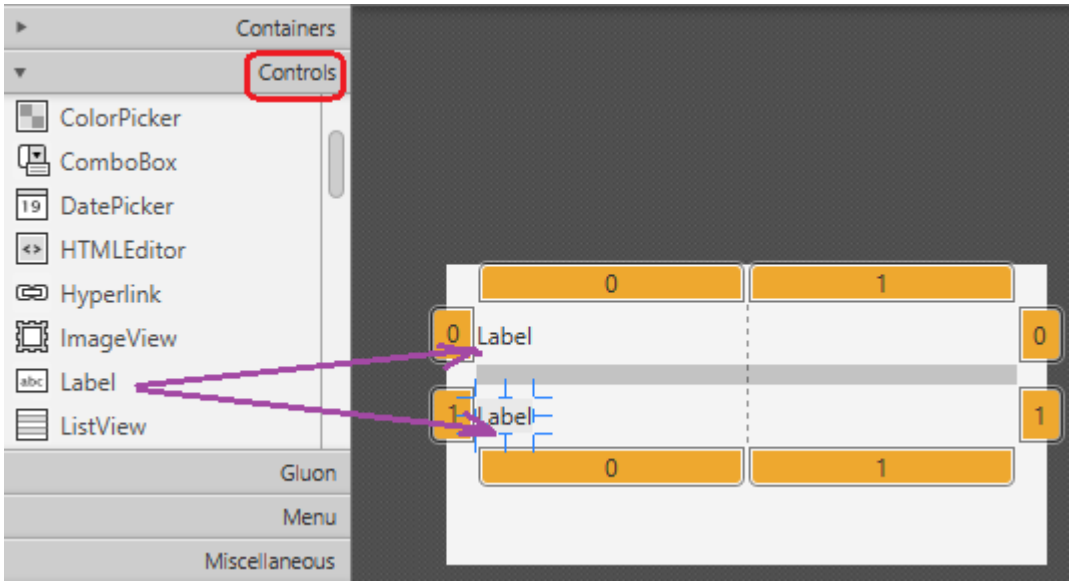


Рисунок 17. Размещение компонентов Label

Зададим текст и установим шрифт через свойства **`Text`** и **`Font`** в правой панели на вкладке **`Properties`** (рис. 18).

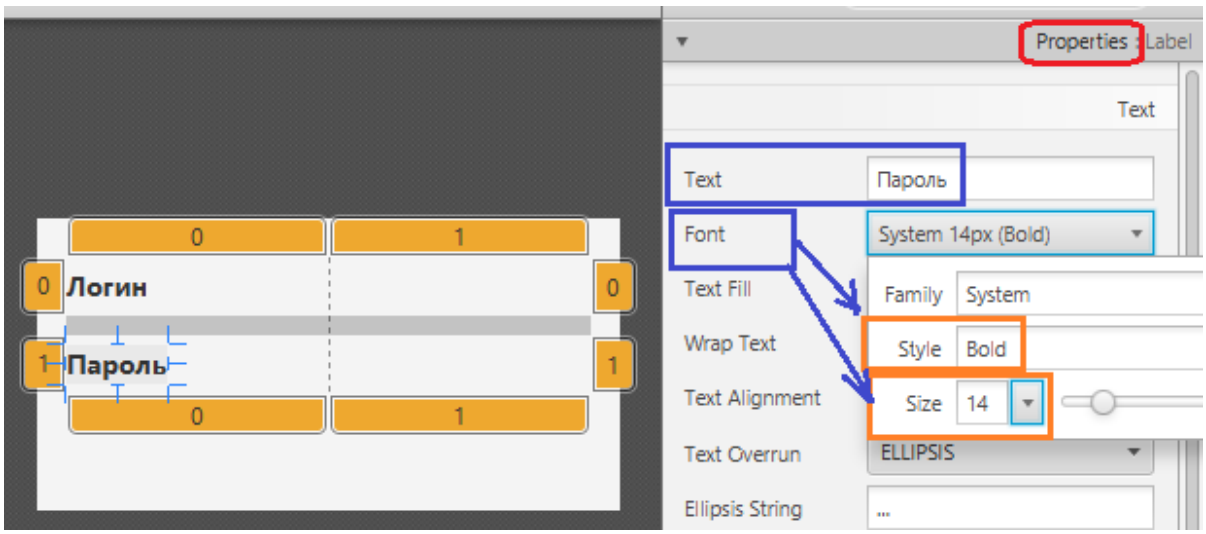


Рисунок 18. Изменение текста и шрифта компонента Label

Разместим в ячейках второй колонки контейнера сетки компонент **`TextField`** для ввода логина и компонент скрытого ввода **`PasswordField`** (рис. 19).

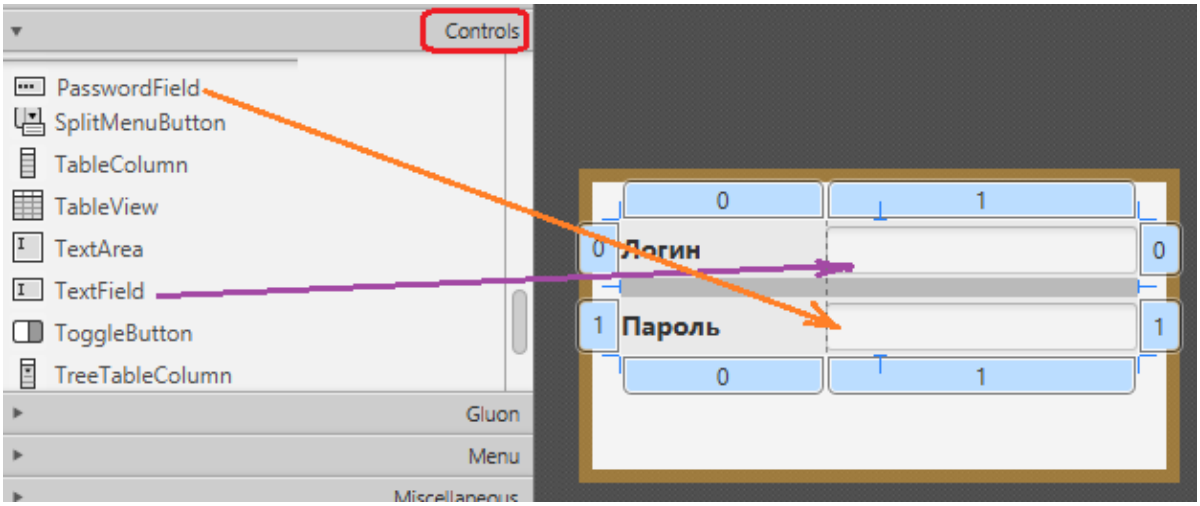


Рисунок 19. Размещение компонентов поля-ввода

Разместим в нижней части корневой панели AnchorPane две кнопки. Зададим для кнопок текст надписи (свойство Text) и установим «жирный» шрифт размера 14(рис.20).

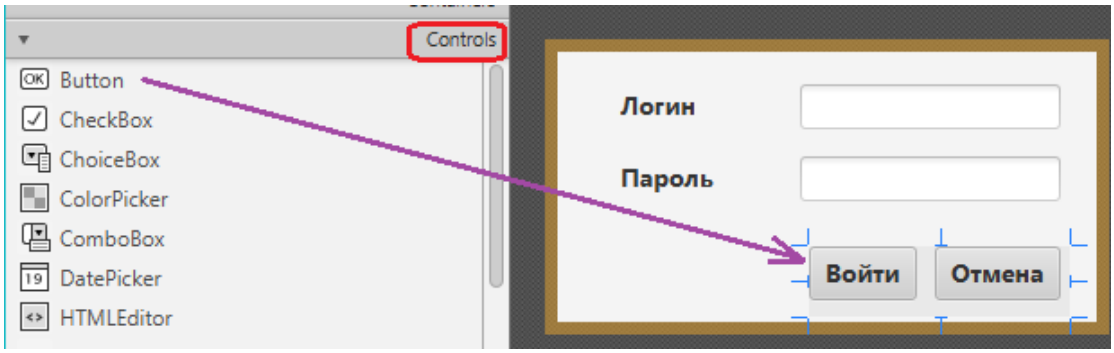


Рисунок 20. Размещение кнопок

Поместим кнопки в контейнер **HBox**. Для этого выделим кнопки и в контекстном меню выберем **Wrap in – Hbox** (рис.21).

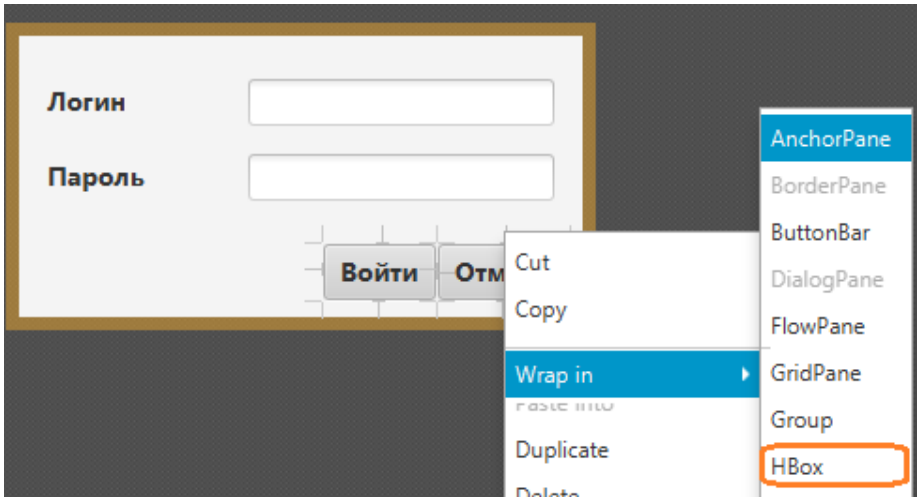


Рисунок 21. Включение кнопок в контейнер HBox

Зададим для контейнера HBox правую границу 20 и расстояние между ячейками `Spacing`=10 (рис. 22).

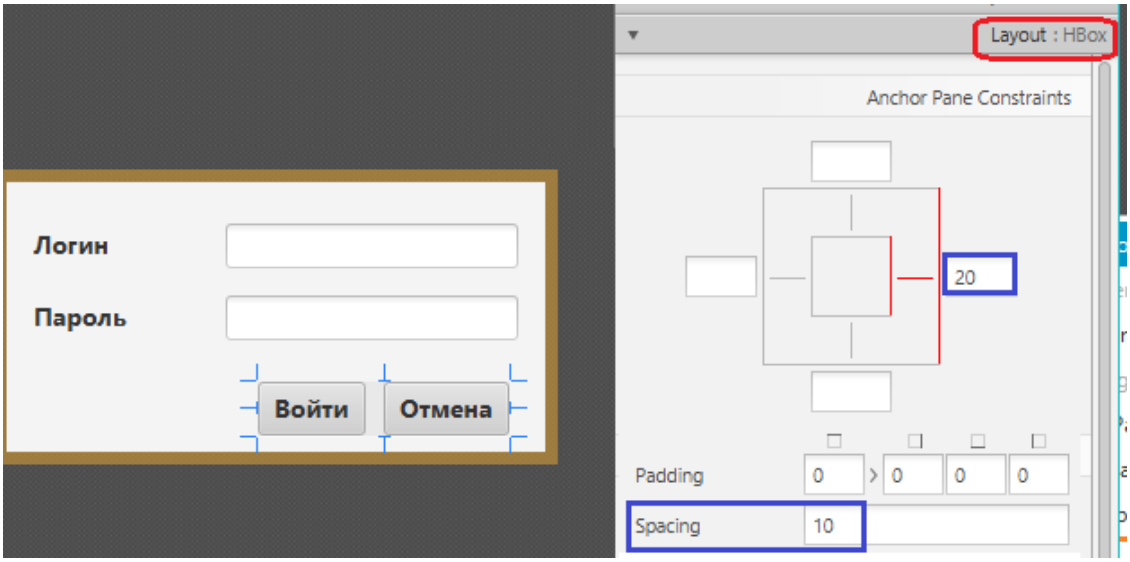


Рисунок 22. Задание отступов для контейнера HBox

Окончательный вид формы представлен на рисунке 23. На вкладке **Hierarchy** видны все компоненты пользовательского интерфейса. Заголовок и иконку окна авторизации зададим в программном коде.

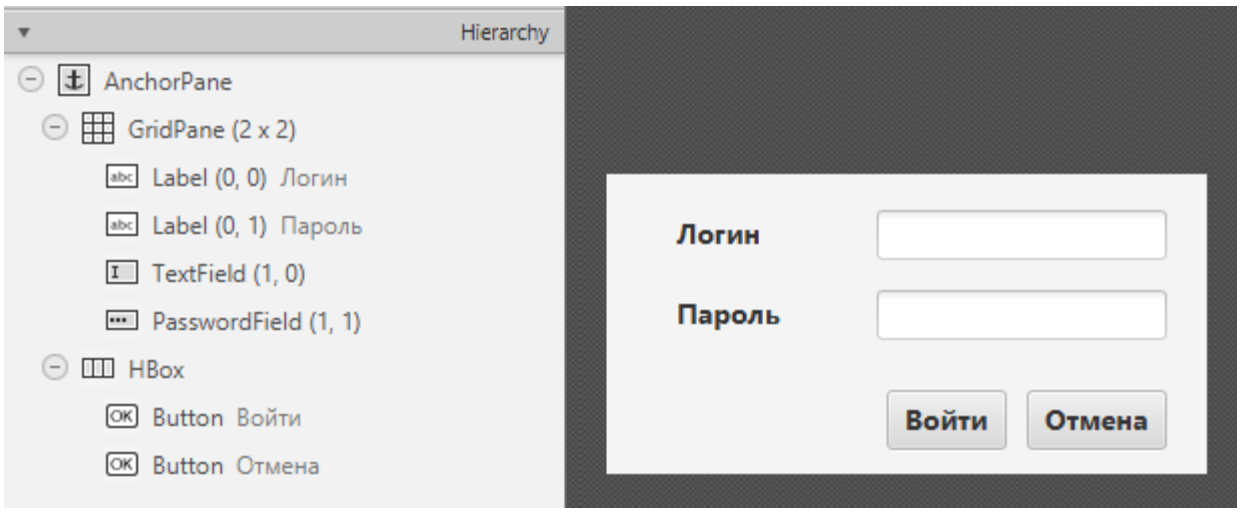


Рисунок 23. Окончательный вид окна авторизации

На этом завершим создание пользовательского интерфейса окна авторизации. Все изменения необходимо сохранить через пункт меню

`File-Save`

Перейдем к созданию класса контроллера. Класс **`LoginDialogController`** будем создавать в пакете **`com.controllers`** (рис. 24).

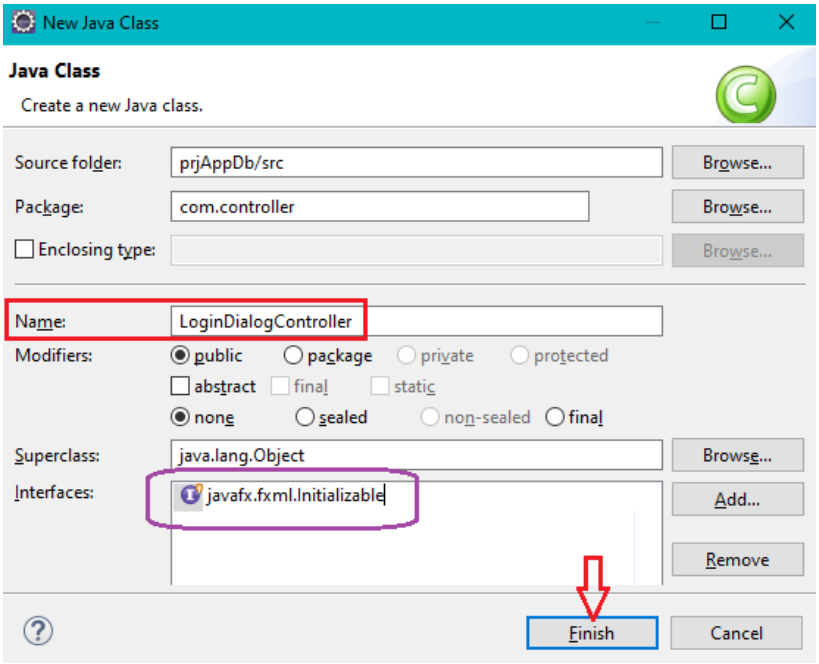


Рисунок 24. Создание класса контроллера

После создания класса компилятор может указать на ошибку. В этом случае необходимо следуя подсказке компилятора внести предлагаемые изменения в конфигурационный файл **`module-info.java`** (рис.25).

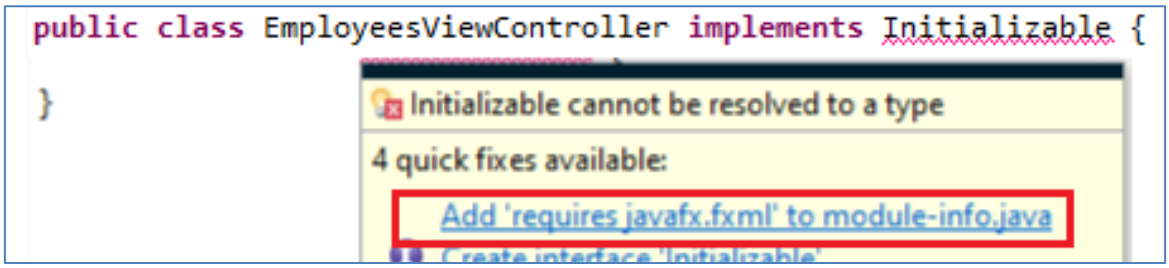


Рисунок 25. Внесение изменений в файл module-info.java

В начале класса контроллера добавим поля

```
public class LoginDialogController implements Initializable {
      @FXML
      private TextField edLogin;
      @FXML
      private PasswordField edPwd;
      private Stage dialogStage;
      private boolean okClicked = false;
      private DBManager manager;
      private String url_db;
```

Пояснения

1. Поля интерфейса для ввода логина **edLogin** и пароля **edPwd** помечены аннотацией **@FXML**. Она необходима для того, чтобы fxml-файл имел доступ к полям и методам с модификатором доступа private. Позже внесем изменения в файл fxml.
2. Поле пароля **edPwd** создается компонентом **PasswordField**, позволяющим скрывать вводимые символы.
3. Поле **dialogStage** представляет окно в приложении JavaFX пользовательского интерфейса окна
4. Булевское поле **okClicked** будет получать логическое значение в зависимости от успешности установки соединения. Это необходимо для запускного класса для принятия решения о продолжении работы приложения или его завершении
5. Поле `manager` содержит ссылку на объект класса взаимодействия с БД. После установки соединения объект соединения будет храниться в объекте `manager` и использоваться для выполнения запросов.
6. Поле `url_db` содержит строку соединения с БД, значение которой загружается из файла свойств.

Далее добавим ряд методов. Методы, вызов которых указан в файле fxml, должны иметь аннотацию @FXML.

1) Метод инициализации окна авторизации

```
public void initialize(Stage dialogStage, DBManager manager) {
      // Установка объекта Stage окна
      this.dialogStage = dialogStage;
      this.manager = manager;
      // Чтение файла свойств проекта
```

```
	Properties properties= new Properties();
	try {
		// Загрузка файла свойств
		// Файл свойств находится в корневой папке проекта
		// Чтение строки соединения из файла свойств
		File file = new File("conf.prop");
		properties.load(new FileReader(file));
		url_db = properties.getProperty("URL_DB");
		// Чтение логина и пароля из файла свойств
		// и запись их а поля ввода
		edLogin.setText(properties.getProperty("User"));
		edPwd.setText(properties.getProperty("Pwd"));
	} catch (IOException e) {
		System.err.println("ОШИБКА: Файл свойств отсутствует!");
	}
}
```

Важное пояснение

Обратите внимание, что поля класса контроллера, относящиеся к компонентам интерфейса (в данном классе – это поля ввода логина и пароля **edLogin** и **edPwd**) не создаются конструктором явно в программе. Они автоматически создаются средствами JavaFX. Если создать их конструктором в программе, то это будут объекты, недоступные в интерфейсе окна.

2) Метод чтения поля **okClicked**

```
public boolean isOkClicked() {
	return okClicked;
}
```

3) Обработчик нажатия кнопки «Войти» с аннотацией **@FXML**.

```
@FXML
private void handleOk() {
	okClicked = false;
	if (authenticate()) {
	   okClicked = true;
	}
	dialogStage.close();
}
```

4) Обработчик нажатия кнопки «Отмена» с аннотацией **@FXML**

```
@FXML
private void handleCancel() {
	okClicked = false;
	dialogStage.close();
}
```

5) Метод авторизации

```
protected boolean authenticate() {
	String login = null;
	char[] password = null;
	try {
		login = edLogin.getText().trim();
```

```
            password = edPwd.getText().toCharArray();
            if (login.length() == 0 || password.length == 0)
                  throw new Exception("Логин и/или пароль не указаны.");
      } catch (Exception e) {
            Dialogs.showDialog("Ошибка ввода данных",
                        e.getMessage(),AlertType.ERROR);
                  edPwd.setText(null);
                  return false;
      }
      // пробуем авторизоваться
      try {
            String pwd = new String(password);
            Connection con = connect(login, pwd);
            if (con == null)
                  throw new Exception("Логин,пароль или имя БД указаны не верно");
                  manager.setConnection(con);
                  System.out.println(manager.getVersion());
            } catch (Exception e) {
                  Dialogs.showDialog("Ошибка соединения с БД",
                        e.getMessage(),AlertType.ERROR);
                  edPwd.setText(null);
                  return false;
            }
            return true;
      }
```

6) Метод соединения с БД

```
private Connection connect(String log, String pass) {
      Locale.setDefault(Locale.ENGLISH);
      try {
            // Данная строка для разных БД может отличаться
            // Для PostgreSQL, Oracle, MySQL, MSSQL
            Connection con = DriverManager.getConnection(url_db, log, pass);
            con.setAutoCommit(false);
            return con;
      } catch (SQLException e) {
            e.printStackTrace();
            return null;
      }
}
```

Связываем класс контроллера с файлом описания интерфейса. Для этого внесем изменения в файл **LoginDialog.fxml**, открыв его в приложении Scene Builder.

Прежде всего, укажем класс контроллера для пользовательского интерфейса окна авторизации. Для этого на левой панели во вкладке **Controller** в поле **Controller class** введем имя контроллера с указанием его положения в проекте (рис. 26):

`com.controllers.LoginDialogController`

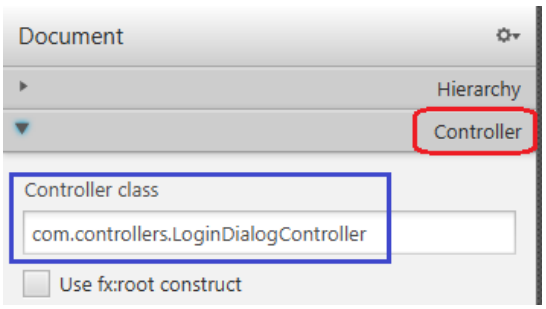


Рисунок 26. Задание класса контроллера

Также зададим имена полям ввода логина и пароля (имена должны совпадать с именами полей в классе контроллера). Для этого на правой панели во вкладке **`Code`** в поле **`fx:id`** введем имена из класса контроллера **`edLogin`** и **`edPwd`** (рис. 27).

И, наконец, зададим для кнопок методы-обработчики, описанные в классе контроллере. Для этого выделим поочередно кнопки и на правой панели во вкладке **`Code`** в поле **`on Action`** введем имена методов из класса контроллера, соответственно **`handleOk`** и **`handleCancel`** (рис. 28).

На этом создание окна авторизации завершено.

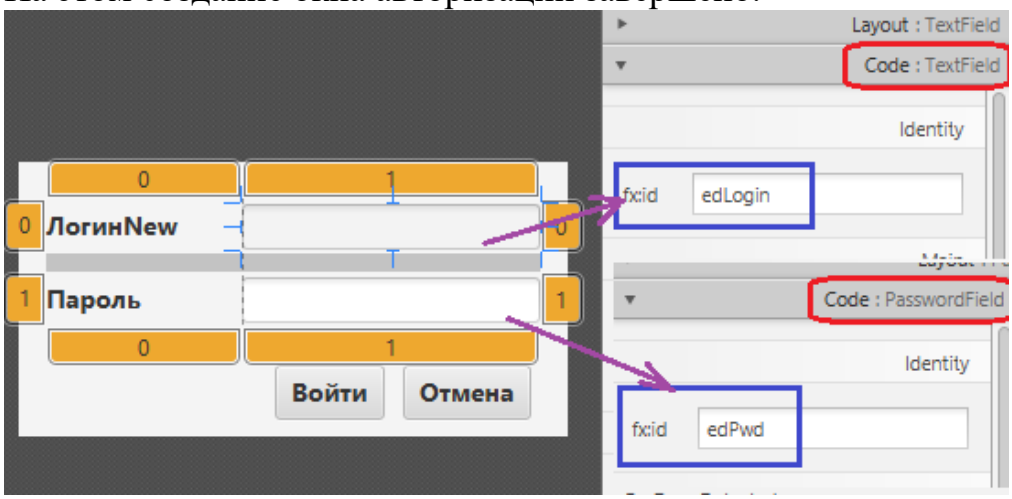


Рисунок 27. Задание имени полям ввода

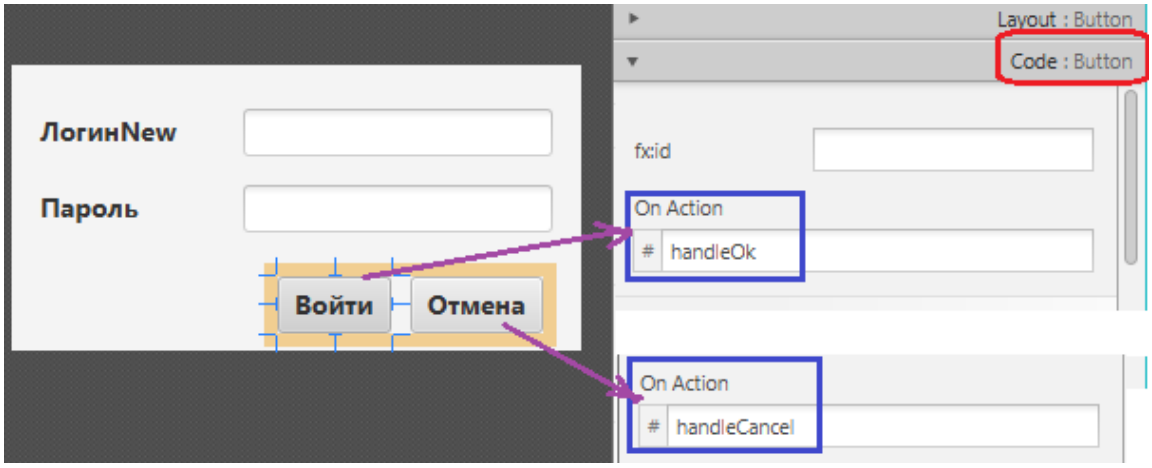


Рисунок 28. Задание обработчиков для кнопок

В начало

7. Создадим основное окно приложения `MainWindow`

Перейдем к созданию главного окна приложения (рис. 29). На данном этапе добавим в это окно меню, компонент изображения, а также зададим заголовок и иконку окна.

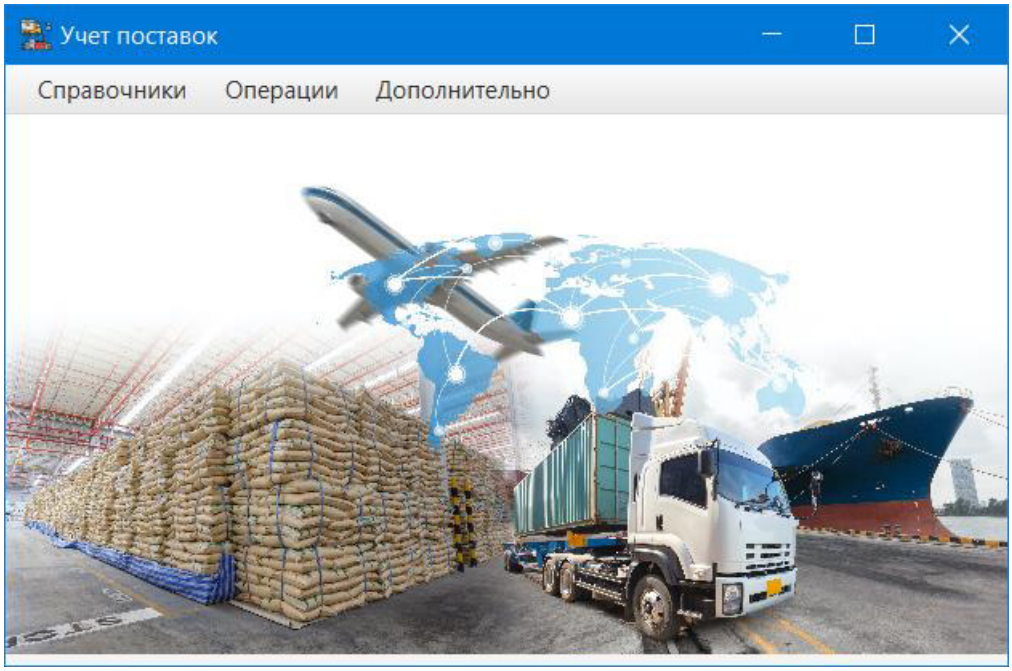


Рисунок 29. Главное окно приложения

Создадим файл fxml **`MainWindow.fxml`** и класс контроллера **`MainWindowController`**. Файл fxml будем размещать в папке **`com.fxml`**, а класс контроллера – в пакете **`com.controllers`**.

Вначале откроем файл **`MainWindow.fxml`** в Scene Builder.

Зададим размер корневой панели 500x400 пикселей. Перетащим с вкладки **`Controls`** на корневую панель **`AnchorPane`** компоненты основного меню **`MenuBar`** и изображения **`ImageView`**.

Для компонента изображения зададим размер 500x375 и отступы **`top`**=25, **`left`** и **`right`**=0. Изображение будем загружать в классе контроллера. Изображение должно иметь размер близкий к размеру компонента **`ImageView.`**

Для компонента основного меню зададим отступы **`top,left`**=0. В компонент основного меню с вкладки **`Menu`** добавим 3 компонента подменю **`Menu`** и пункт меню **`MenuItem`**, как показано на рисунке 30. В данной работе будем добавлять обработчик только к пункту меню **`Выход`**.

Примечание. Если в компоненте основного меню будут уже компоненты **`Menu`** и пункты меню **`MenuItem`**, то надо просто изменить их названия.

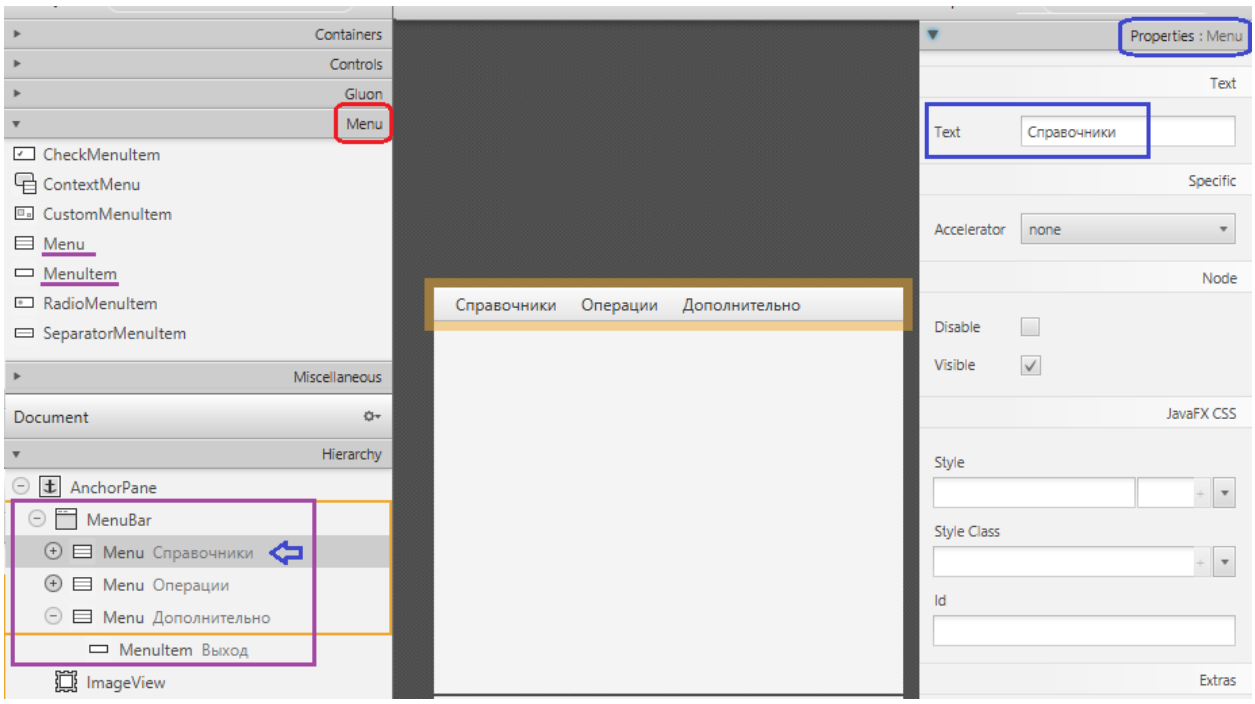


Рисунок 30. Структура основного меню

Обратимся теперь к классу контроллера **`MainWindowController`**.

```
public class MainWindowController implements Initializable {
	@FXML
	private ImageView imgView;
	private DBManager manager;
	private Stage dialogStage;

	@Override
	public void initialize(URL arg0, ResourceBundle arg1) {
	}

	@FXML
	public void initialize(Stage dialogStage, DBManager manager) {
		// Устанавливает сцену для этого окна
		this.dialogStage = dialogStage;
		// Загрузка изображения в ImageView
		// заменить имя файла изображения на свое
		InputStream imgStream =
		  getClass().getResourceAsStream("/images/postavki.jpg");
		Image image = new Image(imgStream);
		imgView.setImage(image);
		// Инициализация объекта manager
		this.manager = manager;
	}

	// Обработчик пункта меню Выход
	@FXML
	private void handleExit() {
		dialogStage.close();
	}
}
```

Пояснения

1) Компонент изображения **imgView** помечен аннотацией **@FXML**. Аннотация необходима для того, чтобы fxml-файл имел к нему доступ для загрузки изображения.
2) Файл с изображением (в коде **postavki.jpg**) у вас будет другим и имя файла надо заменить на свое.
3) Поле **manager** содержит ссылку на объект класса взаимодействия с БД. Объект передается из основного (запускного) класса и инициализирует поле **manager**. При вызове других окон приложение объект **manager** будет передаваться им также для выполнения запросов к БД.
4) Все изображения находятся в папке **images**.

Внесем изменения в файл **MainWindow.xml**, открыв его в приложении Scene Builder.

Прежде всего, укажем класс контроллера для пользовательского интерфейса главного окна приложения. Для этого на левой панели во вкладке **Controller** в поле **Controller class** введем имя контроллера с указанием его положения в проекте:

```
com.controllers.MainWindowController
```

Также зададим имя компонента изображения **ImageView**. Для этого на правой панели во вкладке **Code** в поле **fx:id** введем имя **imgView**.

И, наконец, зададим метод-обработчик для пункта меню **Выход**, описанный в классе контроллере. Для этого выделим пункт меню **Выход** и на правой панели во вкладке **Code** в поле **on Action** введем имя метода **handleExit**.

На этом создание главного окна приложения завершено.

В начало

8. Изменим основной класс `MainApp` (точка входа в приложение)

Основной (запускной) класс приложения **`MainApp`** реализуется следующие действия. Вначале выводится окно авторизации. Если соединение будет установлено, то выводится главное окно приложения. В противном случае приложение завершает свою работу.

Внесем изменения в описание класса. Удалим все поля класса и методы за исключением методов `main` и `start`. В методе `start` удалим весь имеющийся код. Далее внесем изменения, в соответствии с приведенным ниже листингом.

```
package com;

public class MainApp extends Application {
      private DBManager manager = null;

      // Метод старта приложения
      @Override
      public void start(Stage arg0) throws Exception {
            // Вывод окна авторизации
            if (showLoginDialog()) {
                  // При успешном соединении отображение главного окна
                   showMainWnd();
            } else {
                  // закрытие приложения
                  closeApplication();
            }
      }

      // Вывод окна авторизации
      public boolean showLoginDialog() {
            manager = new DBManager();
            try {
                  // Загружаем fxml-файл и создаём новую сцену
                  FXMLLoader loader = new FXMLLoader();
                  loader.setLocation(MainApp.class.getResource(
                        "fxml/LoginDialog.fxml"));
                  AnchorPane page = (AnchorPane) loader.load();
                  // Создаём диалоговое окно Stage.
                  Stage dialogStage = new Stage();
                  // Задаём заголовок окна
                  dialogStage.setTitle("Авторизация");
                  dialogStage.initModality(Modality.WINDOW_MODAL);
                  Scene scene = new Scene(page);
                  dialogStage.setScene(scene);
                  // Загрузка иконки (имя файла с иконкой
                  //  надо изменить на свое!!!)
                  InputStream iconStream =
                        getClass().getResourceAsStream(
                        "/images/connect48.png");
                  if (iconStream == null)
                        System.out.println("Файл не найден");
                  else {
                        Image image = new Image(iconStream);
                        dialogStage.getIcons().add(image);
```

```
                }
                // Связываем контроллер с окном
                LoginDialogController controller =
                     loader.getController();
                // Инициализация контроллера
                controller.initialize(dialogStage,manager);
                // Вывод окна с ожиданием его закрытия
                dialogStage.showAndWait();
                return controller.isOkClicked();
            } catch (IOException e) {
                e.printStackTrace();
                return false;
            }
        }

        private void showMainWnd() {
            try {
                // Загружаем fxml-файл главного окна
                FXMLLoader loader = new FXMLLoader();
                loader.setLocation(MainApp.class.getResource(
                           "fxml/MainWindow.fxml"));
                AnchorPane page = (AnchorPane) loader.load();
                // Создаём диалоговое окно Stage.
                Stage dialogStage = new Stage();
                // Задаём заголовок окна
                dialogStage.setTitle("Учет поставок");
                dialogStage.initModality(Modality.WINDOW_MODAL);
                Scene scene = new Scene(page);
                dialogStage.setScene(scene);
                // Загрузка иконки (имя файла с иконкой
                //  надо изменить на свое!!!)
                InputStream iconStream =
                getClass().getResourceAsStream("/images/post.png");
                Image image = new Image(iconStream);
                dialogStage.getIcons().add(image);
                // Передаём адресата в контроллер.
                MainWindowController controller = loader.getController();
                // Инициализация контроллера
                controller.initialize(dialogStage,manager);

                // Отображаем диалоговое окно и ожиданием его закрытия
                dialogStage.showAndWait();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
        // Закрытие приложения
        private void closeApplication() {
             System.exit(0);
        }
        public static void main(String[] args) {
             launch(args);
        }
}
```

Пояснения. При создании окна авторизации и основного окна им назначаются иконки. Все изображения находятся в папке **`images`**. В классе

используется изображение `post.png, connect48.png.` В вашем случае иконки будут другими и имена иконок в программном коде надо заменить на свои.

Протестируем приложение. Поставим курсор на класс запуска **`MainApp`** и выполним в контекстном меню пункт

`Run As Java Application`

или нажмем кнопку **`Выполнить (Run)`** на панели инструментов.

Примечание. При запуске программы может быть выдано сообщение об ошибке с текстом

```
...
java.base/java.lang.Thread.run(Thread.java:833)
Caused by: java.lang.IllegalAccessException: class
javafx.fxml.FXMLLoader$ValueElement (in module javafx.fxml) cannot access class
com.controllers.LoginDialogController (in module com) because module com does
not export com.controllers to module javafx.fxml
```

Для исправления ошибки необходимо в конце конфигурационного файла `module-info.java` (в пакете `com`) добавить строку:

```
opens com.controllers;
```

Это необходимо для того, чтобы загрузчик файлов `fxml` получил доступ к пакету с классами контроллеров (`package com.controllers`).

После запуска программы появится окно авторизации пользователя, в котором введем имя пользователя и пароль.

Если все выполнено правильно, то будет выведено главное окно приложения.

В начало

9.* Создадим исполняемый `jar`-файл

Файл используется для запуска приложения вне среды Eclipse.

Выполним следующие действия.

a) Обновим проект

В дереве объектов `Project Explorer` (или `Package Explorer`) сделаем правый клик на строке проекта и в контекстном меню выберем (рис. 31):

`Maven – Update Project`

Рисунок 31. Обновление проекта

b) Выполним сборку проекта

Сделаем правый клик на строке проекта и в контекстном меню выберем (рис. 32):

`Run as – Maven build`

Рисунок 32. Сборка проекта

В окне сборке в поле **`Goal`** введем (рис. 33):

`clean verify`

и нажмем **`Run`**.

Запустится процесс сборки и при нормальном завершении процесса в консоли будет выведено **`BUILD SUCCESS`** (рис.34).

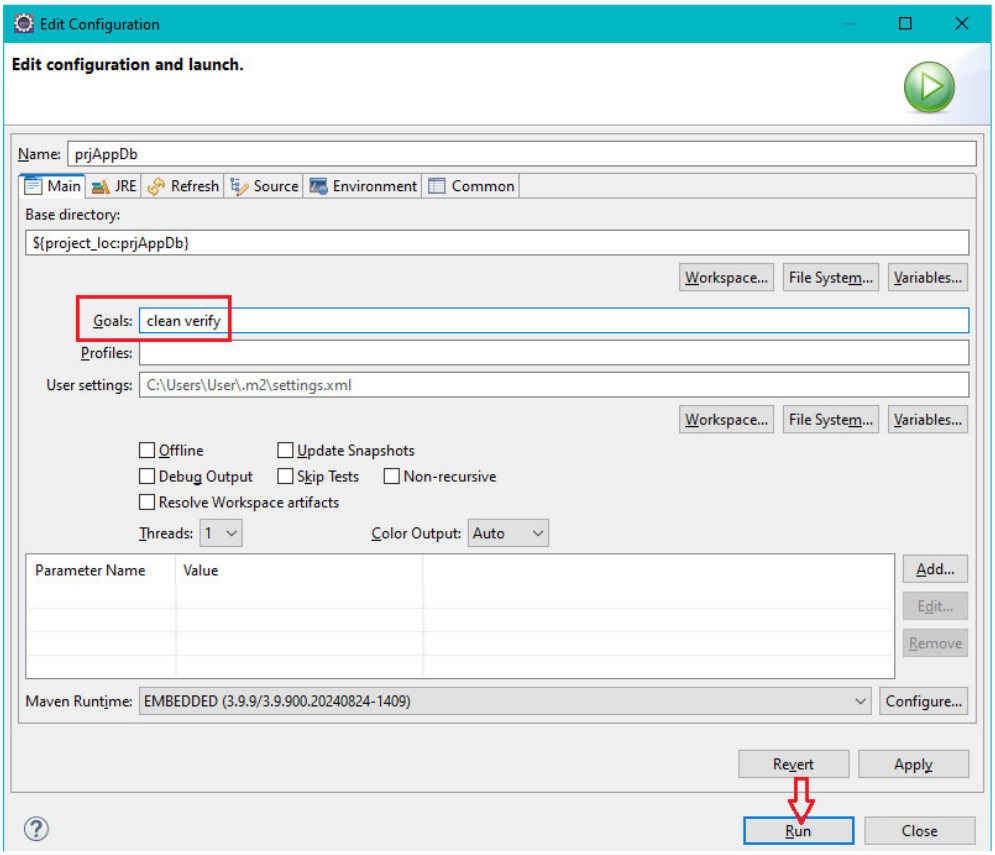


Рисунок 33. Параметры сборки проекта

```
[INFO] No tests to run.
[INFO]
[INFO] --- jar:3.4.1:jar (default-jar) @ prjAppDbMvn ---
[INFO] Building jar: D:\ECLIPSE\eclipse-jee-2024-06-R-win32-x8
[INFO] ------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------
[INFO] Total time:  3.248 s
[INFO] Finished at: 2024-11-17T01:46:21+03:00
[INFO] ------------------------------------------------------------
```

Рисунок 34. Успешное завершение процесса сборки

c) Создадим исполняемый jar-файл*

Выделим строку проекта и выполним команду меню **`Export`** (рис. 35). В окне экспорта выберем тип экспорта **`Runnable JAR file`** (рис. 36).

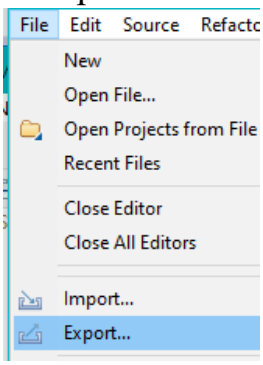


Рисунок 35. Экспорт проекта

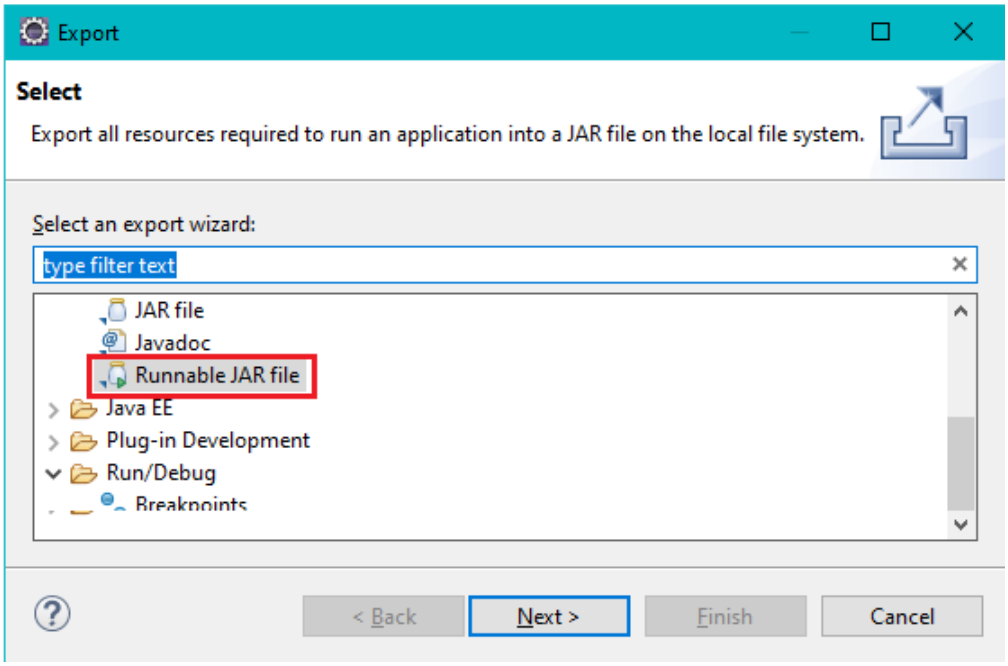


Рисунок 36. Выбор типа экспорта проекта в исполняемый jar-файл

В следующем окне выберем (рис. 37):

- конфигурацию исполняемого файла
- место расположения исполняемого файла
- способ экспорта библиотек

и нажмем `Finish`.

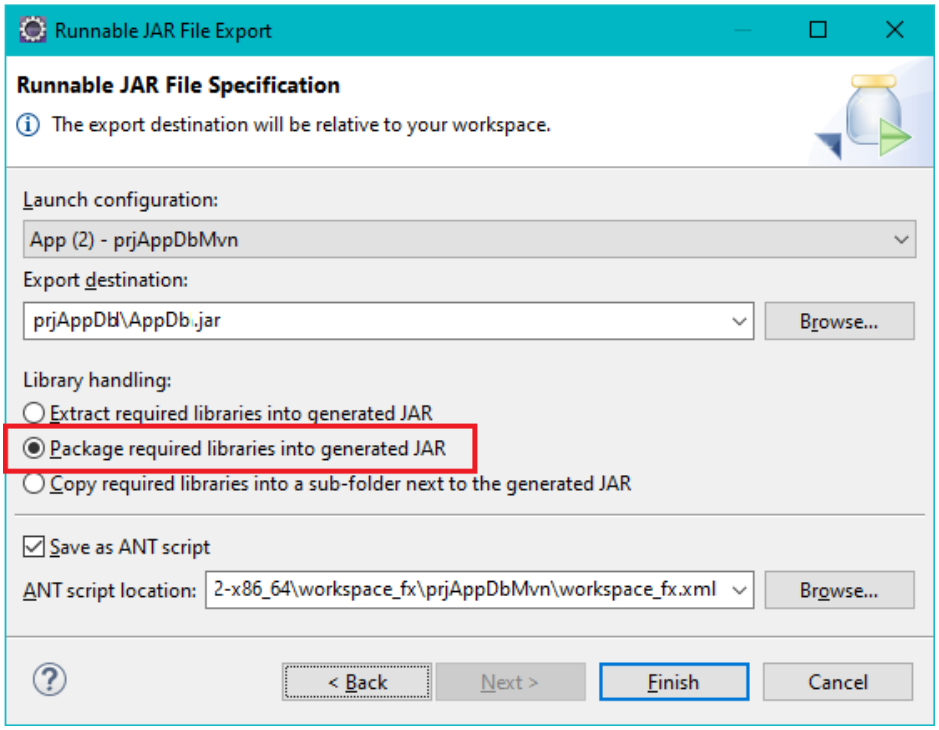


Рисунок 37. Выбор места расположения исполняемого jar-файла

В результате будет создан исполняемый файл `AppDb.jar.` Положим рядом с jar-файлом файл настроек `conf.prop` и запустим программу двойным щелчком по имени файла в проводнике.

Запуск файла можно также выполнять в менеджерах командной строки (например, cmd) командой

```
java –jar AppDb.jar
```

В начало